

Date: / /

(HFN)

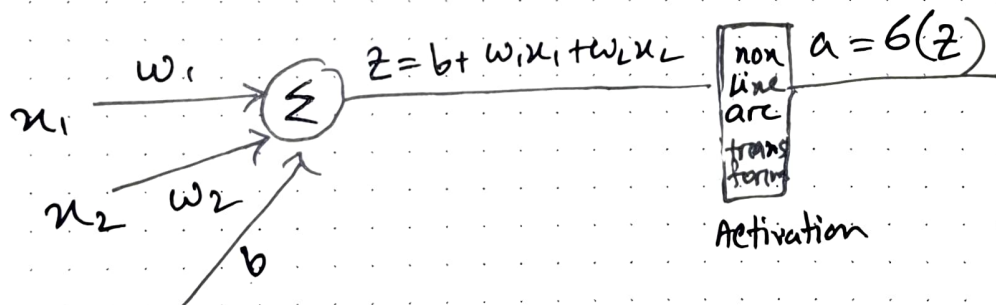


Logistic Regression

→ Refer to slides + book for mathematical derivation and overall intuition.

Logistic Regression is a process to classify a data with finding non-linear correlation to map the linear function to get output for a classification instead of continuous value prediction like linear regression.

Steps



1. initialize weights & biases { weight variable number increases with the feature numbers }
2. Calculate linear output $\Rightarrow z = b + w_1x_1 + w_2x_2$
3. now convert into non linear representation for classification known as "Activation Function" $\rightarrow$ sigmoid func.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad [\text{gives answer from } 0 \rightarrow 1]$$

4. Calculate ~~error~~ ^{loss func.} $\Rightarrow$ log likelihood func.

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Date: / /



here, y_i = truth value, actual label
for a data point.

$\hat{y}_i$ = predicted value (sigmoid func output)
for a data point

n = total number of data points.

5. Calculate error (gradient of loss) & update

* derivations in slide.

* for weights, $\frac{dl}{dw_i} = \left[\sum_{i=1}^n (\hat{y}_i - y_i) * x_i \right] / n$

[x_i is with the corresponding weight w_i see the figure above. $\hat{y}_i$ = sigmoid func. output, y_i = truth output, n is the total datapoints]

* for bias, $\frac{dl}{db} = \left[\sum_{i=1}^n (\hat{y}_i - y_i) \right] / n$

* update rule;

$$\left. \begin{aligned} w_i &= w_i - \alpha \frac{dl}{dw_i} \\ b &= b - \alpha \frac{dl}{db} \end{aligned} \right\} \alpha = \text{learning rate}$$

* Repeat step 1 $\rightarrow$ 5.

6. Practice the math from slide 22; while maintaining these steps.

$\Rightarrow$ Here step {1, 2, 3} is known as feed forward / forward propagation in a perceptron

Date: / /

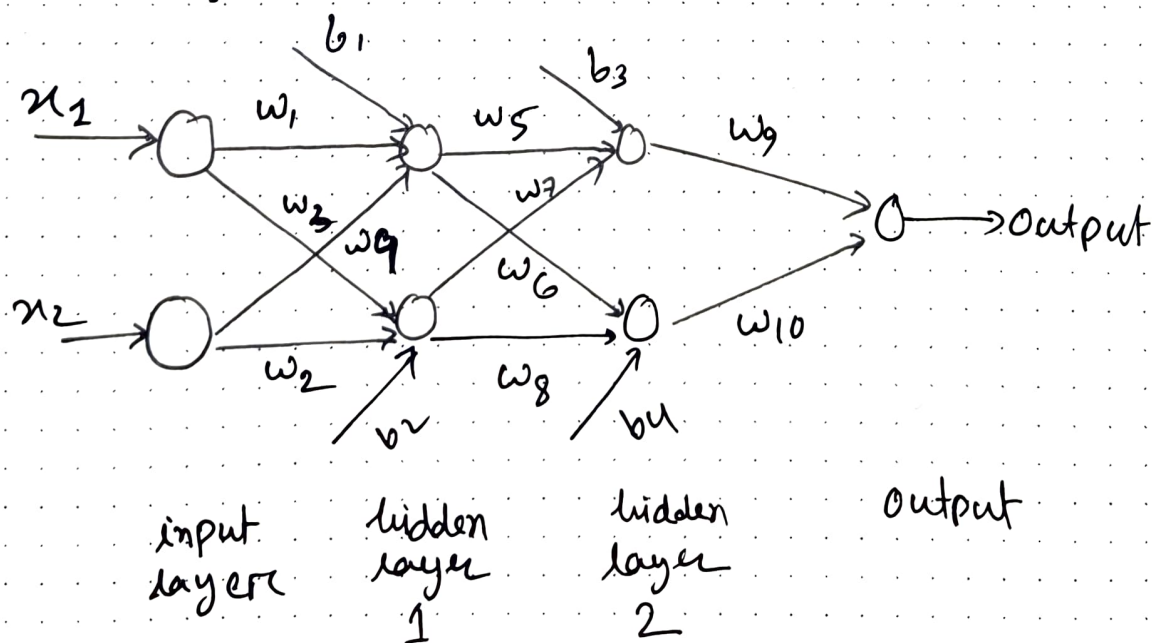
⇒ Step 2,4,5 is called as back propagation.

* The linear regression we did is also known as a single perceptron. Add many of these and get neural network! (sort of)

Neural Network

Just a multilayer perceptron math been done.
Refer to slides + book for proper understanding.

for a binary classification (1 output → Yes/No),
two feature and 2 hidden layer with 2 neuron each
each ⇒



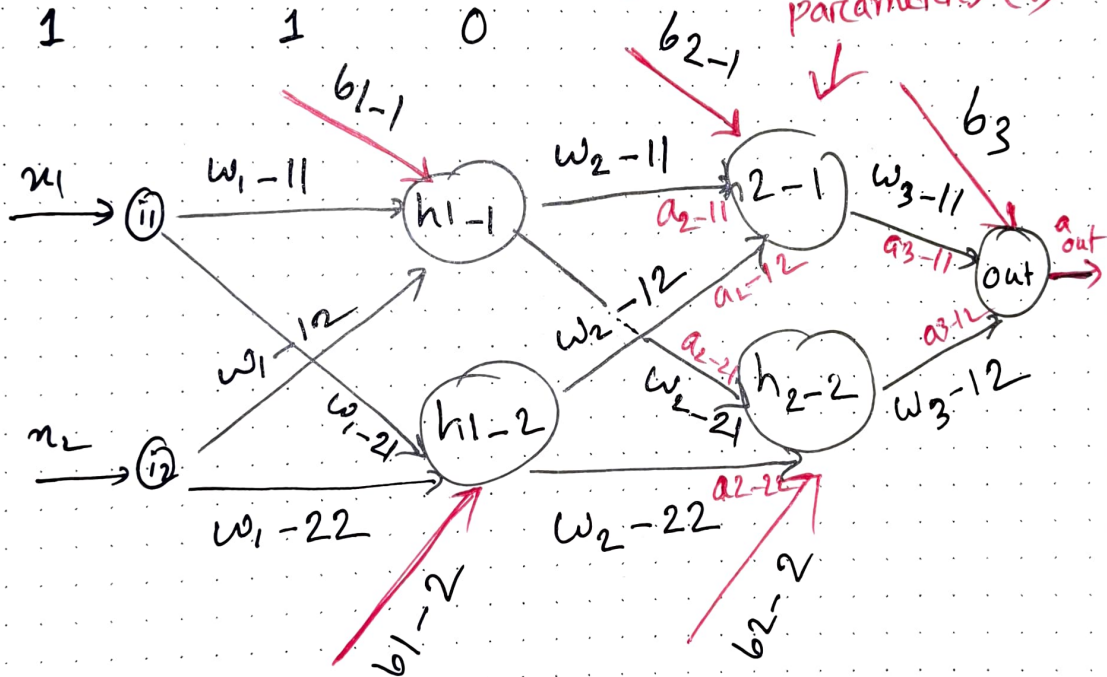
Date: / /



Lets do a math for a simple problem, same setup

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

⊛ here a_{2-1}, a_{2-2} etc are the activation function output from the previous node. So each node from layer 2 onwards take these parameters (a)



naming convention: $w_{\text{layer} - \text{to, from layer layer}}$

example: w_{1-12}

⇒ layer 1 [hidden layer first]
to hidden 1 neuron 1
from input neuron 2

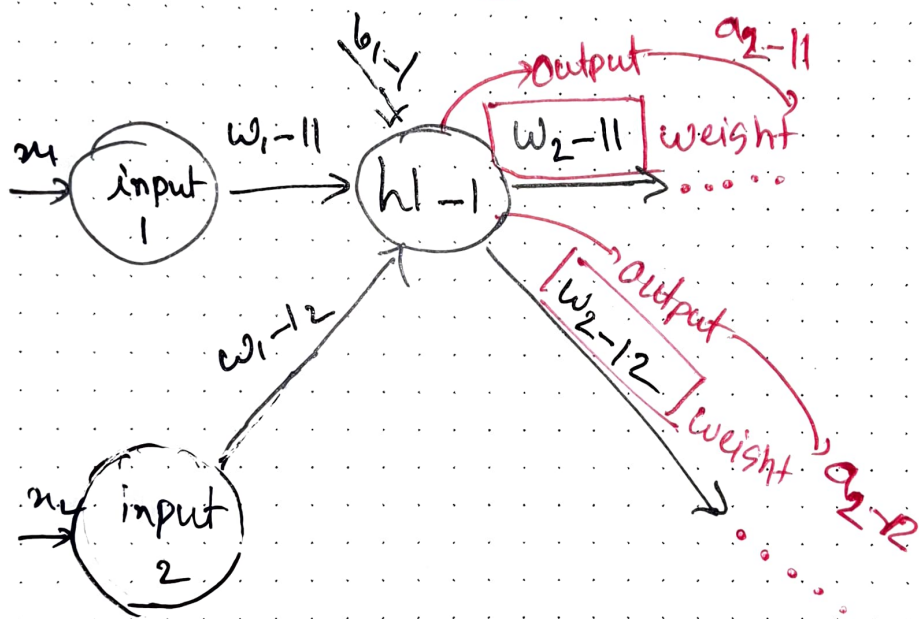
⊛ You can use your own version

Date: / /

Each node after input layer will basically have the same math as log. regre.

activation function $\sigma(z) \Rightarrow$ sigmoid.

* I am isolating only 1 neuron $h1-1$ to show you the calculation. *



for $h1-1$; pre activation (linear value)

$$(z1-1) \Rightarrow \underbrace{w1-11 * x1}_{w1x1} + \underbrace{w1-12 * x2}_{w2x2} + \underbrace{b1-1}_{b1}$$

its nothing but a normal linear prediction as we did before

now, the output from $h1-1$ is $\boxed{a2-11} = \sigma(z1-1)$
Activation function

Date: / /

from the dataset, let's consider data point 2

$$[x_1, x_2] = [0, 1], \quad y = 1$$

let's assume initially all weights are 0.1; all biases are also 0.1.

So, $z_1 = 1$ [linear output] from neuron $h1-1$

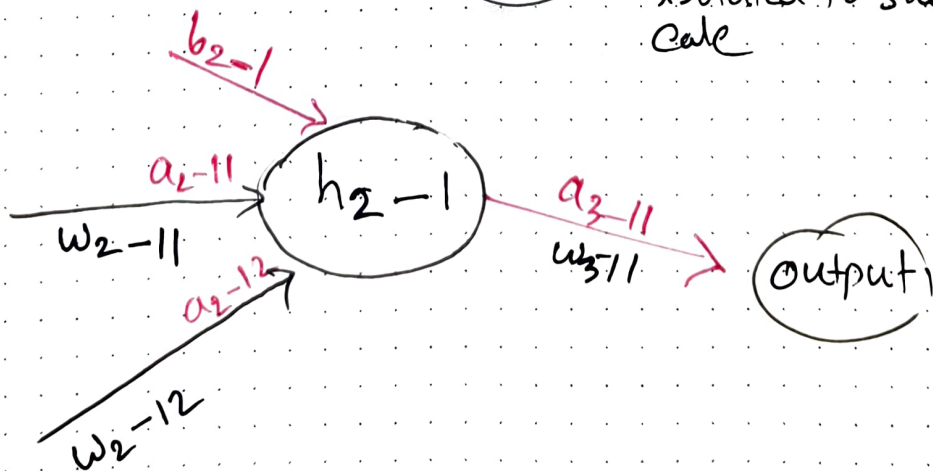
$$\Rightarrow z_1 = 1 = 0.1 * 0 + 0.1 * 1 + 0.1 = 0.2$$

$$\Rightarrow \text{activation} = \sigma(0.2) = 0.55$$

now this is the $a_{2-1} = 0.55$

Similarly, a_{2-12} will also be 0.55.

now, for next node $h2-1$ [next layer] isolated to show calc



$$z_2 = 1 \Rightarrow (w_{2-11} * a_{2-11}) + (w_{2-12} * a_{2-12}) + b_{2-1}$$

[note, for next layers, $\{x_1, x_2, \dots\}$ values are replaced with

Date: / /

the activation func. values of the prev. layer

now, $b(z_2-1) = a_3 - 11 \Rightarrow$ calculate

for all layers we
Now, in the same process, find the activation value of the output layer (a_{out}) which is the prediction value.

Forward Propagation Done. Now calculate errors

$a_{out} \Rightarrow$ predicted. $y_i =$ actual.

For this example, after all calc done,

$$a_{out} = \hat{y}_2 = 0.55; y_{actual} = 1$$

$$\begin{aligned} \text{Loss} = \log \text{likelihood} &= -1 * \log(0.55) \\ &= 0.594 \end{aligned}$$

now back propagation

similar pattern as log. regression. just x_i replaced with the activation of prev. layer

Date: / /

⑤ gradient for output neuron parameters:

$$\rightarrow \frac{dL}{dw_{3-11}} = (\hat{y} - y) * a_{3-11}$$

error
43

↳ notice we always plug x_i here, since for this layer, x_i is the activation output from hidden layer 2, that's what is being put here.

$$\rightarrow \frac{dL}{dw_{3-12}} = (\hat{y} - y) * a_{3-12}$$

error
43

→ error calc same for one neuron.

$$\rightarrow \frac{dL}{db_3} = (\hat{y} - y) *$$

error
43

update accordingly

$$w_{\text{next}} = w_{\text{prev}} - \alpha \text{ gradient}$$

⑥ gradient for layer 2: [hidden layer 2]

for h_{2-1} neuron:

$$\rightarrow \frac{\text{gradient}}{\text{error}} = (\hat{y} - y) * a_{3-11}$$

$$= \left(w_{3-11} * (\hat{y} - y) \right) * a_{3-11} (1 - a_{3-11})$$

chain derivation rule

activation of h_{2-1}

weight * error * activation of h_{2-1} * derivative of sigmoid.

Date: / /

now, plugging this error to layer 2 neuron (n2-1)

$$\frac{dL}{dw_{2-11}} = \frac{\text{error}}{\Delta 2} * a_{2-11}$$

$$\frac{dL}{dw_{2-12}} = \frac{\text{error}}{\Delta 2} * a_{2-12}$$

$$\frac{dL}{db_{2-1}} = \frac{\text{error}}{\Delta 2}$$

⋮

do the same for layer 1. (n1-1)

$$\frac{\text{error}}{\Delta 1} = [w_{2-11} * \Delta 2 + w_{2-21} * \Delta 2] *$$

$$= \left[\text{activation}_{n1-1} (1 - \text{activation}_{n1-1}) \right]$$

now update

$$\frac{dL}{dw_{1-11}}, \frac{dL}{dw_{1-12}}, \dots \text{ accordingly.}$$

Date: / /



Note a pattern:

$$\text{error of a layer} = \left[\text{output weight} * \text{error next layer} \right] *$$

$$\left[\text{activation of cur. layer} * \left(1 - \text{activation of cur. layer} \right) \right]$$